Week 13: MongoDB aggregation and data modelling

Learning outcomes: CLO5

Lesson

An **aggregation pipeline** passes documents through a series of stages:

- `$match` filters documents (like WHERE);
- `$group` groups them and calculates values such as `$sum` (like GROUP BY);
- `$sort` and `$limit` order and cut the result (like ORDER BY and LIMIT);
- `$project` chooses and renames fields;
- `$lookup` joins another collection and puts the matches in an array;
- `$unwind` turns each array element into its own document.

The order of the stages matters. A `$match` near the start means the later stages process fewer documents.

Embedding stores related data inside one document, for example a province with an array of its districts. It is quick to read in one go, but it repeats data and the documents can grow large. **Referencing** stores a code and looks up the related document when needed, like a foreign key. Choose based on how the data is read and updated.

Worked example

Run `example.js` (see Run a MongoDB file (`setup/README.md`)) and read the output. Then open `work.js`.

Collections for this week

The MongoDB database `cambodia` has one document per row of the MySQL tables, with the same field names. MongoDB adds an `_id` to each document. Read from `cambodia` and write only to `student_work`.

Collection	Field	Type	Meaning
`province`	`province_pcode`	string	Province code
`province`	`province_name`	string	Province name (English)
`province`	`area_km2`	number	Area in km², one decimal place
`district`	`district_pcode`	string	District code
`district`	`district_name`	string	District name (English)
`district`	`area_km2`	number	Area in km², one decimal place
`district`	`province_pcode`	string	Province code

Documents are linked by their codes (`district.province_pcode` matches `province.province_pcode`). MongoDB does not check these links the way a MySQL foreign key does.

Fields in the pipelines: group districts by `$province_pcode`. In `example.js`, `$lookup` matches `province_pcode` in both collections and puts the matches in an array called `districts`; after `$unwind`, each district code is `districts.district_pcode`. `district_count` is calculated by `$group`; it is not stored in the data.

Run `db.province.findOne()` in mongosh to see a document. The data guide (`data/README.md`) explains where the data comes from.

Lab tasks

1. Group districts by `province_pcode`, count them and sort by code. Then write a SQL query with `GROUP BY … ORDER BY … LIMIT` and translate it into a pipeline with `$group`, `$sort` and `$limit`. Compare the outputs.
2. Use `$lookup` to show each district with its province name.
3. Use `$lookup` to attach districts to provinces, then `$unwind` them. Compare the number of documents with the row count of the same SQL join.
4. Use `$project` to remove `_id`, and compare the values with a SQL result.
5. Design an embedded model (provinces with their districts) built only from the existing data. Explain how you would keep it up to date, how it would grow and what an update would cost.

Homework

1. Write three pipelines that together use $match, $group, $project, $lookup and $unwind.
2. For two queries in your project, explain whether you would embed or reference, and why.

Capstone sprint

C13: Add three aggregation pipelines with their output, and explain your document model.

Submit

Write your answers in `work.js` and label each one, for example `// Lab 3`. Answer the questions in `reflection.md` and save your query output in `results/`. Then commit and push to your repository. See SUBMISSIONS.md for details.